Final Project

Multi-Agent Customer Support System

A production-ready customer support system built with LangGraph, FastAPI, MCP, LangSmith, and Docker. Three specialist agents, real web search tools, persistent memory, and a streaming REST API.

What It Does

```
POST /chat  {message, thread_id}
↓
Supervisor Agent  →  reads message, routes to the right specialist
↓
Billing Agent      →  invoices, refunds, payments  (web search tool)
Technical Agent    →  bugs, APIs, errors           (web search tool)
General Agent      →  greetings, product info      (no tools)
↓
Streamed response back to user via SSE
↓
GET /history/{thread_id}  →  full conversation history
```

Full Architecture

```
Frontend (curl / React / mobile)
    ↓  HTTP
FastAPI Server  (final_project/api/main.py)
    ■■■ POST /chat          → streams response via SSE
    ■■■ GET /history/{id}   → returns conversation history
        ↓
LangGraph Multi-Agent System  (final_project/graph.py)
    ■■■ Supervisor Agent    → routes based on message content
    ■■■ Billing Agent       → handles billing queries + web search
    ■■■ Technical Agent     → handles technical queries + web search
    ■■■ General Agent       → handles everything else
        ↓  MCP over SSE (production) / stdio (dev)
MCP Tool Server  (final_project/mcp_server/server.py)
    ■■■ web_search()        → Tavily real-time web search
        ↓
```

```
AsyncSqliteSaver            → conversation persistence
LangSmith                   → full observability (traces, costs, latency)
Docker                      → containerized, deployable anywhere
```

File Structure

```
final_project/
■■■ state.py                ← State TypedDict (shared across all agents)
■■■ graph.py                ← assembles the full multi-agent graph
■■■ agents/
■   ■■■ supervisor.py       ← reads message, returns next_agent
■   ■■■ billing.py          ← billing specialist + MCP web search
■   ■■■ technical.py        ← technical specialist + MCP web search
■   ■■■ general.py          ← general assistant, no tools
■■■ mcp_server/
■   ■■■ server.py           ← MCP server with Tavily web_search tool
■■■ api/
■   ■■■ main.py             ← FastAPI app (lifespan, /chat, /history)
■■■ Dockerfile              ← containerizes the FastAPI server
```

Build Order

Step 1: state.py

Define the State TypedDict with messages (add_messages) and next_agent (str). This is the contract every agent reads and writes.

Step 2: mcp_server/server.py

Build the MCP tool server with a real Tavily web_search tool. Test it standalone before connecting to agents.

Step 3: agents/supervisor.py

Supervisor reads the user message and returns next_agent: 'billing_agent', 'technical_agent', or 'general_agent'. Uses extraction loop + safe default.

Step 4: agents/billing.py + technical.py + general.py

Each specialist agent has its own system prompt and connects to the MCP server for web search. Billing and technical agents have tools. General does not.

Step 5: graph.py

Wire all nodes together: START → supervisor → conditional routing → specialist agent → END. Compile with AsyncSqliteSaver checkpointer.

Step 6: api/main.py

FastAPI server with lifespan (open DB + compile graph), POST /chat (streaming SSE), GET /history/{thread_id}.

Step 7: Dockerfile

Containerize the FastAPI server. Test with docker build + docker run. Verify /chat and /history work from inside the container.

Key Design Decisions

Decision	Choice	Why
Persistence	AsyncSqliteSaver (dev) → PostgresSaver (prod)	SQLite = zero setup. Swap one line for Postgres in production.
MCP transport	stdio (dev) → SSE (prod)	stdio = simple, no infra. SSE = separate container, scalable.

Streaming	astream() + StreamingResponse + SSE	Users see tokens as they arrive. Standard production pattern.
Supervisor routing	Extraction loop + safe default	LLMs don't follow exact format. Always validate, always have a fallback.
Observability	LangSmith via env vars	Zero code change. Captures every node, token, tool call automatically.
Containerization	Docker with --host 0.0.0.0	Runs identically anywhere. The most common Docker mistake avoided.

Build Phases

Phase 1 — Core System (runnable, interview-ready)

- state.py — State TypedDict with messages + next_agent
- mcp_server/server.py — Tavily web_search tool over stdio
- agents/ — supervisor, billing, technical, general
- graph.py — full multi-agent graph with conditional routing
- api/main.py — FastAPI with /chat (SSE streaming) + /history
- AsyncSqliteSaver — conversation persistence
- LangSmith — observability via env vars
- Dockerfile — containerized FastAPI server

Phase 2 — Production Infrastructure

- Swap AsyncSqliteSaver → AsyncPostgresSaver
- Swap MCP stdio → SSE transport, MCP server as separate container
- docker-compose.yml — run FastAPI + MCP server together
- Deploy to Railway / Render / AWS — live URL

Phase 3 — Advanced Patterns

- Human-in-the-loop interrupt before billing agent sends refunds
- Auto-retry from checkpoint for failed agent runs
- Per-agent MCP tool servers (billing-tools-server, technical-tools-server)

This is not a tutorial project. This is the stack companies actually run.